Fejlesztő adatai:

- Név: Ballai Péter Lóránt
- Neptun kód: EYSEED

1. Feladatleírás

Készítsünk programot, amellyel az aknakereső játék kétszemélyes változatát játszhatjuk. Adott egy n*n mezőből álló tábla, amelyen rejtett aknákat helyezünk el. A többi mező szintén elrejtve tárolja, hogy a velük szomszédos 8 mezőn hány akna helyezkedik el. A játékosok felváltva léphetnek. Egy mezőre lépve felfedjük annak tartalmát. Ha az akna, a játékos veszített. Amennyiben a mező nullát rejt, akkor a vele szomszédos mezők is automatikusan felfedésre kerülnek (és ha a szomszédos is nulla, akkor annak a szomszédai is, és így tovább). A játék addig tart, amíg valamelyik játékos aknára nem lép, vagy fel nem fedték az összes nem akna mezőt (ekkor döntetlen lesz a játék). A program biztosítson lehetőséget új játék kezdésére a pályaméret megadásával (6*6, 10*10, 16*16), valamint játék mentésére és betöltésére. Ismerje fel, ha vége a játéknak, és jelenítse meg, melyik játékos győzött (ha nem döntetlen).

2. **UML Felhasználói Esetek (Use Case)**

A feladat elemzése: A játékot egy 6*6, 10*10 vagy 16*16 méretű négyzetrácsos táblán játssza két játékos (1. játékos és 2. játékos). Új játék indításakor a program véletlenszerűen elhelyez (n^6)/6 darab aknát a táblán, majd minden mezőre előre kiszámítja a 8 szomszédos mezőn található aknák számát. Kezdetben minden mező rejtett.

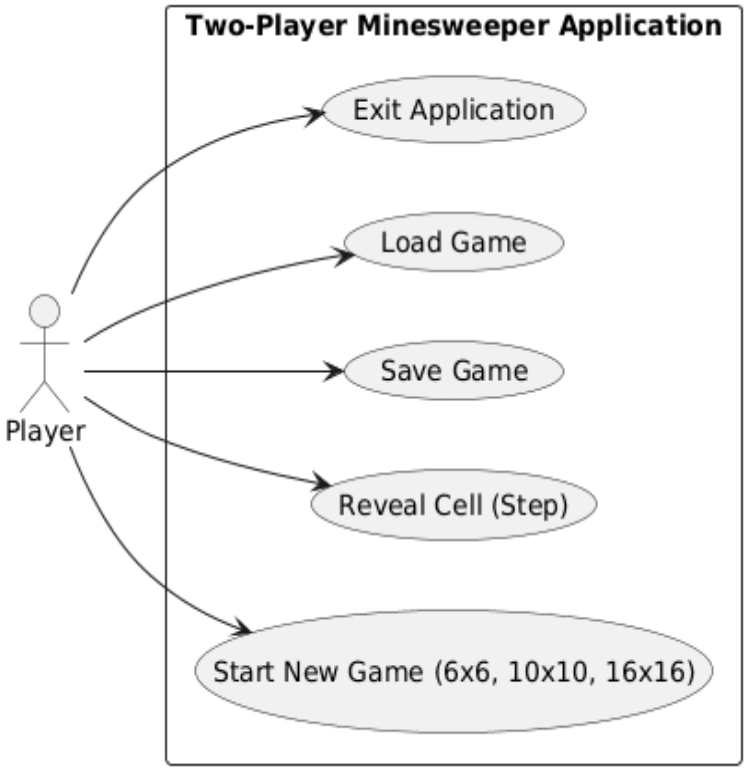
A játékosok felváltva kattintanak egy-egy még fel nem fedett mezőre. A kattintás hatására három eset lehetséges:

- **Akna mező felfedése:** Az aktuális játékos aknára lépett, így azonnal veszít, és a másik játékos nyeri a játékot. A program felfedi az összes aknát a táblán, és megjeleníti a győztes nevét.
- **Szomszédos aknával rendelkező mező (1–8) felfedése:** Csak az adott mező válik láthatóvá és jeleníti meg a szomszédos aknák számát, majd a kör átkerül a másik játékoshoz (amennyiben maradt még fel nem fedett biztonságos mező).
- **Üres (0 szomszédos aknájú) mező felfedése:** Szélességi bejárás (BFS / Flood Fill) segítségével automatikusan felfedésre kerül az adott mező és minden vele összefüggő 0-s terület, beleértve a területet határoló számozott mezőket is.

Ha egy lépés eredményeként az összes biztonságos (nem akna) mezőt felfedték a játékosok anélkül, hogy bárki aknára lépett volna, a játék döntetlennel ér véget.

Felhasználói esetek:

- **Új játék indítása (6x6, 10x10, 16x16):** A felhasználó a menüből kiválasztja a tábla méretét, mire a program véletlenszerűen legenerálja az aknákat, alaphelyzetbe állítja a játékot és a táblához igazítja az ablakméretet.
- **Mező felfedése (Lépés):** A soron következő játékos rákattint egy még rejtett mezőre a táblán.
- **Játékállás mentése:** A felhasználó fájlba mentheti a tábla méretét, a soron következő játékost és a mezők állapotát (akna-e, felfedett-e, szomszédos aknák száma).
- **Játékállás betöltése:** A felhasználó egy korábban elmentett fájlból visszatöltheti a félbehagyott játékot.
- **Kilépés:** A felhasználó bezárja az alkalmazást.



3. **A Porgram szerkezete**

Az alkalmazás háromrétegű architektúrában épül fel: megjelenítési réteg (Minesweeper), üzleti logikai réteg (Minesweeper.Model) és perzisztencia réteg (IMinesweeperDataAccess, MinesweeperDataAccess).

Perzisztencia réteg

- **IMinesweeperDataAccess** (interfész): Meghatározza a játékállás aszinkron betöltéséhez (LoadAsync) és mentéséhez (SaveAsync) szükséges műveleteket, biztosítva a modell réteg függetlenségét és tesztelhetőségét.
- **MinesweeperDataAccess** (osztály): Megvalósítja az adatelérési interfészt szöveges fájlok segítségével (StreamReader, StreamWriter). A fájl első sora a táblaméretet és az aktuális játékost tartalmazza, a további sorok pedig a tábla mezőit akna:felfedett:szomszédok formátumban tárolják.

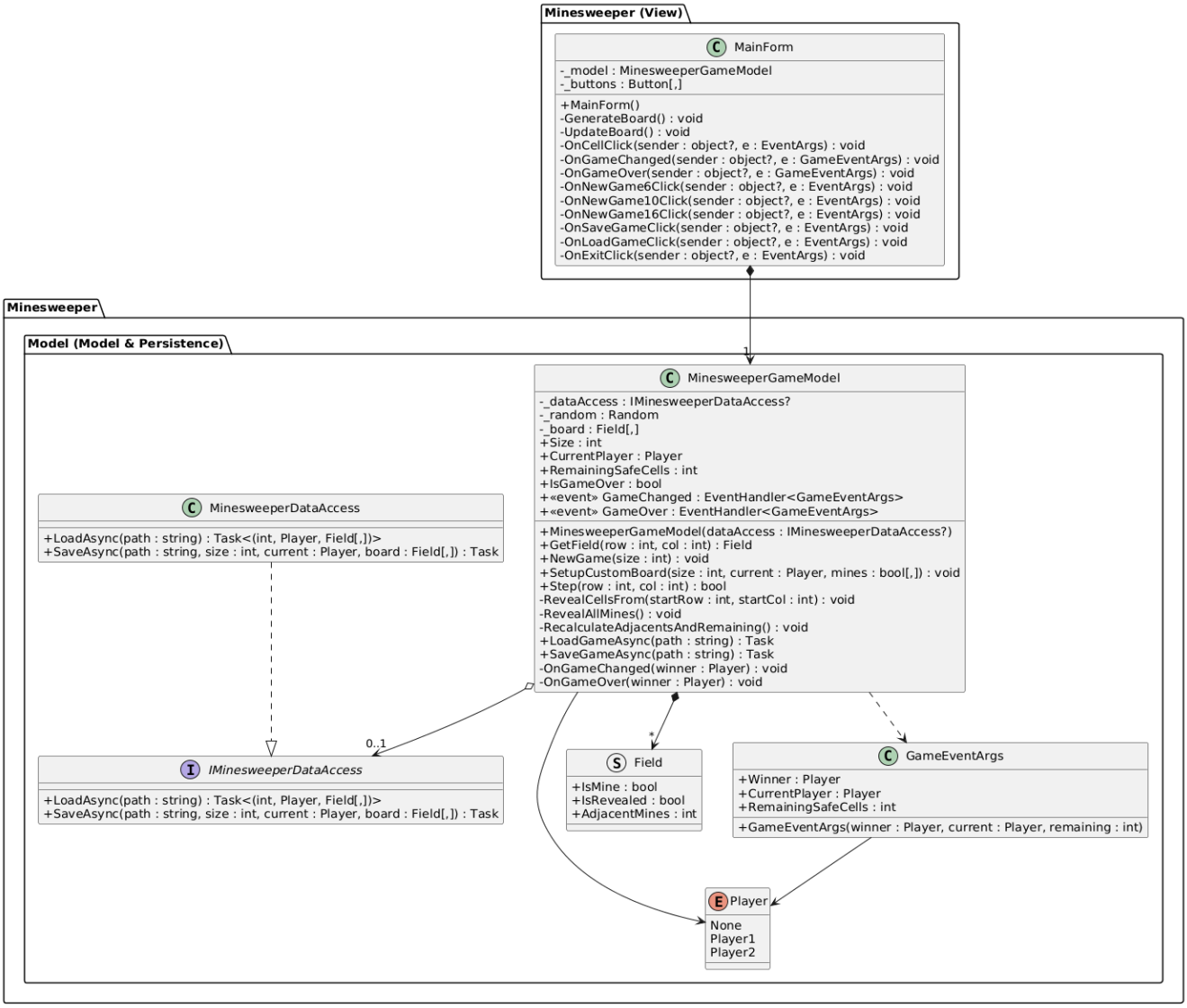
Modell réteg (Minesweeper.Model)

- **Player** (enum): A játékosokat azonosítja (None, Player1, Player2).
- **Field** (struktúra): Egyetlen mező állapotát tárolja: tartalmaz-e aknát (IsMine), fel van-e fedve (IsRevealed), illetve hány akna van a szomszédságában (AdjacentMines).
- **GameEventArgs** (osztály): Az eseménykezeléshez használt adatosztály, amely továbbítja a győztest (Winner), a soron lévő játékost (CurrentPlayer) és a még hátralévő biztonságos mezők számát (RemainingSafeCells).
- **MinesweeperGameModel** (osztály): A játék fő üzleti logikáját tartalmazó osztály.
- Tárolja a tábla mezőit (_board), a méretet (Size), az aktuális játékost (CurrentPlayer), a hátralévő biztonságos mezők számát (RemainingSafeCells) és a játék végét jelző logikai értéket (IsGameOver).
- A **NewGame**(int size) metódus véletlenszerűen elhelyezi az aknákat, majd a RecalculateAdjacentsAndRemaining() segédmetódussal kiszámítja a szomszédos aknák számát.
- A **Step**(int row, int col) metódus hajtja végre a mező felfedését: ellenőrzi az aknára lépést, szükség esetén a RevealCellsFrom metódussal (sor alapú szélességi bejárással) felfedi az összefüggő 0-s mezőket, ellenőrzi a döntetlent, és kiváltja a GameChanged, illetve GameOver eseményeket.

Nézet réteg (Minesweeper)

- **MainForm (osztály):** A Windows Forms főablak, amely tartalmazza a menüsort (MenuStrip), az állapotsort (StatusStrip) és a futási időben dinamikusan felépített nyomógomb-mátrixot (Button[,]).

- Feliratkozik a modell eseményeire (OnGameChanged, OnGameOver), a felfedett mezők gombjait letiltja (Enabled = false), megjeleníti rajtuk a szomszédos aknák számát vagy az aknát (*), és a táblaméret változásakor automatikusan átméretezi az ablakot.



4. Tesztesetek

A modell működését automatizált MSTest egységtesztek ellenőrzik a MockDataAccess segédosztály használatával:

1. **NewGame_InitializesBoardCorrectly:** Paraméterezett teszt, amely ellenőrzi, hogy új játék indításakor a tábla mérete, a kezdőjátékos és a biztonságos mezők száma megfelelően inicializálódik-e.
2. **Step_ZeroAdjacentMines_CascadesReveal:** Ellenőrzi, hogy egy 0 szomszédos aknával rendelkező mezőre lépve a program automatikusan felfedi-e az összes összefüggő üres és határoló számozott mezőt.
3. **Step_HitMine_OtherPlayerWins:** Ellenőrzi, hogy ha az 1. játékos aknára lép, a játéknak vége szakad-e, és a GameOver esemény a 2. játékost hirdeti-e ki győztesnek.
4. **Step_RevealAllSafeCells_ResultsInDraw:** Ellenőrzi, hogy az összes nem-akna mező felfedésekor a játék döntetlennel (Player.None) ér-e véget.
5. **Step_AlreadyRevealedCell_ReturnsFalseAndKeepsTurn:** Ellenőrzi, hogy egy már felfedett mezőre való újbóli kattintás érvénytelen lépésnek minősül-e (false), és nem adja-e át a kört a másik játékosnak.
6. **SaveAndLoadGame_UsesDataAccessCorrectly:** A szimulált adatelérésen keresztül ellenőrzi a játékállás mentésének és betöltésének helyességét.